


# INF2006 Team Project 1 Report

# FlyWise –

Cloud-Based Flight Delay Prediction & Intelligence Application

Team: P2 Group 12

Team Members:

| Student ID | Name | Email |
|---|---|---|
| 2502122 | Chong Xin Huei | 2502122@sit.singaporetech.edu.sg |
| 2500837 | Damien Teh Zhanrong | 2500837@sit.singaporetech.edu.sg |
| 2501690 | Daniel Tay Zhu Hao | 2501690@sit.singaporetech.edu.sg |
| 2503049 | Wong Jinghong | 2503049@sit.singaporetech.edu.sg |
| 2500024 | Wong Zheng Sheng, Jasper | 2500024@sit.singaporetech.edu.sg |

# 1. Problem, Users and Success Criteria

## 1.1 Problem

Although flight passengers are aware of their scheduled flight departure and arrival times, they have limited access to the likelihood, time, and severity of delays to their flights. This is particularly relevant in the U.S. domestic aviation industry, where nearly 1 in 4 flights did not arrive on time in 2025 (Yahoo Finance, 2026). Additionally, an analysis of flight data estimated that U.S. travellers collectively lost approximately 1.5 million hours to flight delays, or 171 years in 2025, highlighting the significant impact delays can have on passengers (SkyRefund, 2026). With flight delays occurring frequently and resulting in substantial passenger time loss, there is a need for a data-driven approach that can identify patterns and factors associated with delays and provide passengers with more meaningful and accurate information about their potential flight-delay risk.

## 1.2 Users

Airline passengers and travellers seeking information about their potential flight delays.

## 1.3 Success Criteria

FlyWise's success will be assessed through a repeatable flight-information or saved-flight workflow, rejection of invalid inputs, interpretable departure-delay outputs and evaluation against a stated baseline. Cloud operation will be assessed through dated endpoint checks, a documented scaling observation and an operational monitoring record. The expected workload of 50–100 requests per minute is a design assumption requiring load testing; response-time and availability targets have not yet been validated. Security assessment must distinguish implemented controls from unresolved least-privilege and public-transport limitations.

# 2. Solution Overview and Architecture

## Solution overview

FlyWise combines live flight-delay risk prediction and historical flight analytics in a web application. React frontend is integrated with FastAPI backend running in Docker on AWS EC2. Users access the application through an internet-facing Application Load Balancer, which forwards requests to the application instances on port 8000. The compute tier uses an Auto Scaling Group with a CPU target of 40%.

The backend queries the Amazon RDS MariaDB database for historical statistics, user accounts and saved flights. The AWS RDS uses a Multi-AZ deployment, with a primary instance and standby in separate private subnets. The backend also calls the Machine Learning (ML) XGBoost model through the predict endpoint to generate delay-risk estimates on Aviationstack’s supplied live flight information.

A private Amazon S3 bucket stores the historical flight-data CSV. The available documentation confirms storage but does not establish an automated ingestion connection from S3. CloudWatch provides infrastructure metrics and alarms supporting the scaling policy.

The system architecture diagram shows the application components and their interactions. The cloud architecture diagram shows the AWS deployment, network boundaries and availability design.

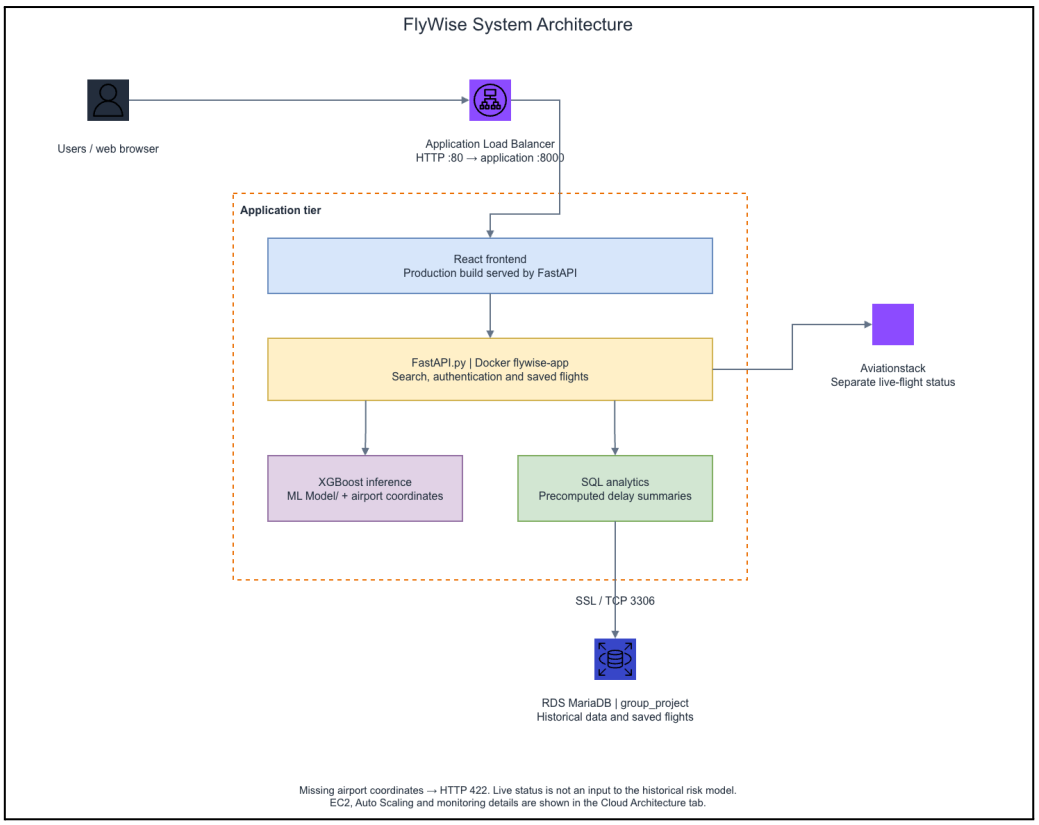



*Figure 1: System Architecture Diagram*

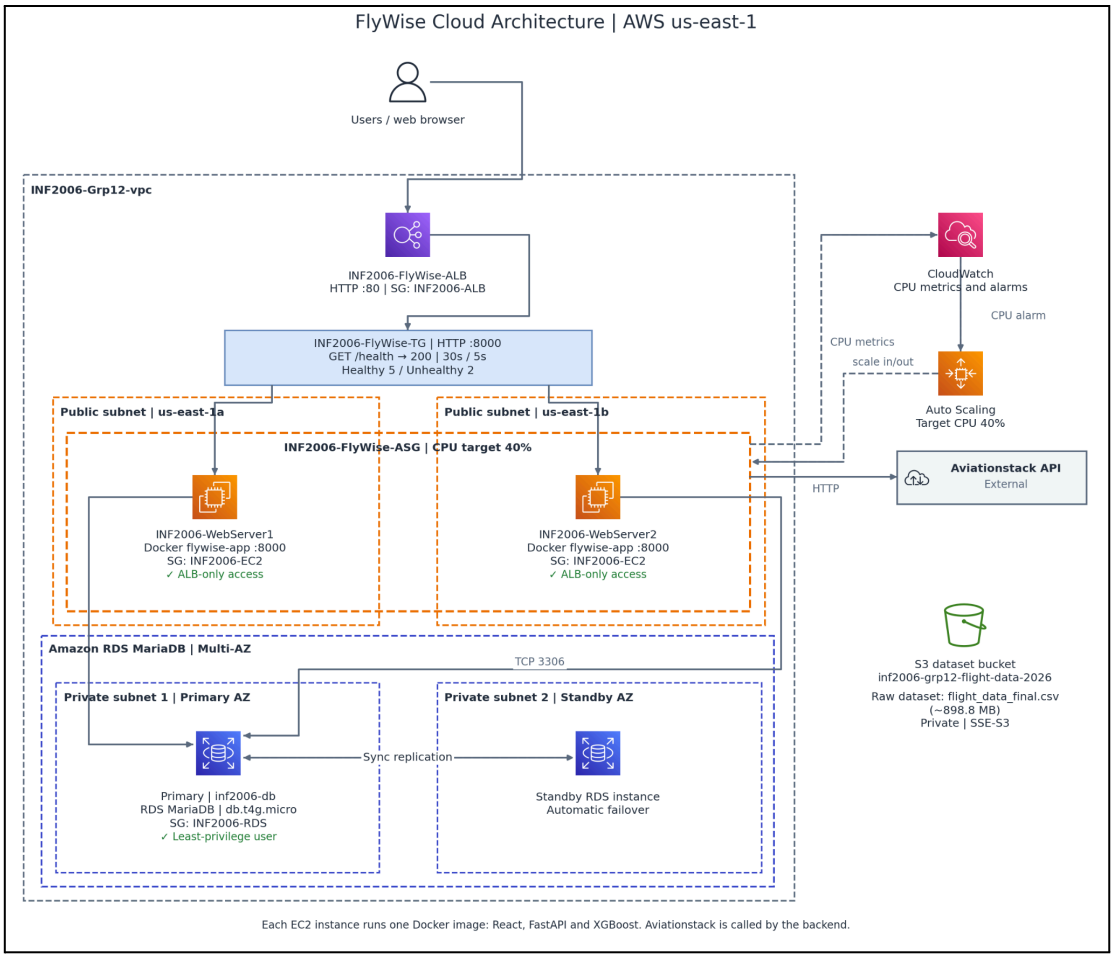



*Figure 2: Cloud Architecture Diagram*

### 2.1 Key Assumptions

- Geographic scope: The historical dataset and trained model concern U.S. commercial flight operations represented in the BTS-derived dataset. Predictive performance outside that domain has not been established. This boundary applies to the historical model; the separate live flight-tracking feature may return flights outside that domain where supported by Aviationstack.
- Temporal Scope & Historical Baseline: The predictive risk model is bound to historical operational patterns (e.g., seasonal headwinds, carrier-specific route bottlenecks, time-of-day air traffic congestion) derived from BTS data. A separate, non-predictive live flight-tracking feature (via a third-party flight-status API) supplements this for flights that are already scheduled and within the live-tracking provider's coverage window; this live data is not used as model input and does not affect the ML-generated risk score.

### 2.2 Constraints

- The application is deployed on small EC2 instances to control development and evaluation costs. Memory, CPU capacity, and external API request limits therefore constrain the workload it can support. Prediction coverage is also limited by the airport-coordinate file required to calculate flight distance. Flights involving airports absent from this file cannot receive a prediction.
- Public application access currently uses HTTP. Database least privilege and verification of ALB-only access to EC2 remain outstanding in the available documentation. These limitations must be distinguished from implemented security controls.

### 2.3 Expected Workload

- The initial design assumption is approximately 50–100 requests per minute across flight searches, analytics and prediction requests. This represents expected request throughput rather than simultaneous user concurrency. Load testing is required to establish whether the deployed application meets this workload and to measure response times and error rates.

## 3. Cloud Service/Deployment Choices and Trade-offs

FlyWise runs on AWS public cloud. EC2 provides IaaS compute, while RDS, ALB and S3 provide managed database, traffic distribution and object storage.

EC2 was retained for control over the operating system, Docker runtime and dependencies, which suits the existing FastAPI app, React build and XGBoost model. The trade-off is responsibility for patching, dependency management and keeping versions consistent across instances.

RDS separates persistent data from replaceable application instances and places a standby in a separate private subnet. Multi-AZ alone does not demonstrate tested failover or backup recovery, which need separate evidence.

The ALB provides a public entry point and health checks, while the Auto Scaling Group uses a 40% CPU target to adjust compute capacity. These services add cost and configuration complexity. S3 stores the historical dataset privately; an automated ingestion path is not established in the available documentation.

Two alternative approaches are relevant to this design. A managed application platform would reduce OS administration but require re-validating the container, model dependencies and database connectivity. Serverless functions would reduce idle compute but require adapting FastAPI and handling model packaging, initialisation time and connection management. Both were rejected to avoid migration work; neither was benchmarked.

## 4. Implementation, data design and security controls

### 4.1 Database schema and data model

FlyWise's persistent data layer is hosted on Amazon RDS running MariaDB (group_project database), with public access disabled and port 3306 restricted to the application tier's security group. The schema includes flight_data and flights for historical records; airlines, airports and routes for reference data; users, saved_flights and upcoming_saved_flights for accounts and saved flights; and route_statistics for precomputed route aggregates. Deployment notes record that route_statistics returned totals of 7,001,619 flights, 1,534,638 delayed flights, 6,938 routes, 352 airports and 14 airlines. Using this table to replace the summary endpoint's full-table query was investigated as a proposed optimisation; the notes do not establish that this change was applied. The delay_cause_summary table (delay_cause, delay_hours, delay_percentage) was created and used by the delay-causes endpoint to avoid repeatedly aggregating millions of flight records. The deployment notes record a local response time of approximately 0.8 seconds after this change; no measured before-and-after comparison was recorded. The documented relational model links flights.airline_id to airlines.airline_id and flights.route_id to routes.route_id. Routes identify their origin and destination through references to airports.airport_id; each origin–destination pair is unique. The flight_id, airline_id, airport_id and route_id columns identify records in their respective tables. A complete schema/migration, including account and saved-flight relationships, must accompany the submission.

### 4.2 Security controls

Deployment documentation records RDS public access disabled, port 3306 restricted to the application tier's security group, and require_secure_transport enabled. These are documented configuration observations rather than a fresh external-access or connection-encryption test. The ALB accepts public HTTP traffic and routes requests to application instances across two Availability Zones; tested service continuity has not been established. FlyWise includes bcrypt password hashing, JWT authentication and user-scoped saved-flight operations, but authentication and cross-user authorisation require recorded tests. Credentials are intended to be supplied through environment variables; correct environment loading and secret-handling checks remain to be verified. Cloud notes also record S3 Block Public Access and SSE-S3 encryption. These storage controls do not establish an automated ingestion path.

### 4.3 Known security limitations

The application currently connects to the database using an administrative account with broad privileges, including CREATE USER and GRANT OPTION. This is a known least-privilege limitation. The planned remediation is to create a dedicated application account with only the table-level permissions required by FlyWise's queries, including DELETE where required by its account or saved-flight workflows, and without DDL or grant privileges. Migration to that account remains pending verification. ALB-only access to EC2 port 8000 also remains to be verified; the intended configuration permits application traffic only from the load balancer's security group.

## 5. Analytics or AI/ML feature: data, method, evaluation and limitations

### 5.1 Dataset provenance

FlyWise uses historical U.S. flight-operation data sourced from the Bureau of Transportation Statistics (BTS), accessed through Kaggle’s "US Flight Delays 2025 (BTS On-Time Performance)" by a7madmostafa. The source dataset is described as a 40-column dataset covering 2025. The final model notebook and analytics notebook both load flight_data_final.csv, recording 7,001,619 rows and 27 retained columns. The analytics outputs record 14 reporting airlines, 352 origin airports and 352 destination airports. After the final model notebook's eligibility and cleaning steps, 6,879,485 rows remain for modelling. The historical records describe flight operations rather than passenger-identifying information.

### 5.2 Target variable

The final model notebook confirms that the classifier predicts dep_del15, renamed delayed for training. The positive class (1) represents a departure delay of at least 15 minutes; class 0 represents a departure delay below that threshold. This is a departure-delay classifier, rather than an arrival-delay classifier. Its output estimates the risk of a late departure and does not directly establish the likelihood of a missed connection or a late arrival.

### 5.3 Preprocessing

The final model notebook removes exact duplicate rows, excludes cancelled and diverted flights, parses flight dates and removes records with invalid dates or missing departure-delay targets. It derives month and day_of_week from flight_date, using pandas' Monday = 0 convention, and extracts departure and arrival hour/minute features from scheduled times. Its time parser accepts HH:MM and HHMM representations and maps hour 24 to midnight. Numeric features use median imputation; reporting airline, origin and destination use most-frequent imputation followed by one-hot encoding, with unknown categories ignored. The preprocessor is fitted on the training set and applied to the held-out test set. The 11 input features become 726 processed features in the recorded run. The field-selection appendix describes storage decisions and should be distinguished from this training pipeline.

### 5.4 Model and Prediction Method

FlyWise uses an XGBoost classifier to produce a flight-delay risk score. The backend loads the saved model, fitted preprocessor and feature-column definition. Its inputs are the reporting airline, origin airport, destination airport, scheduled flight duration, distance, month, day of the week, scheduled departure hour and minute, and scheduled arrival hour and minute.

For each prediction request, the backend derives the calendar and schedule features and calculates distance from airport coordinates, converting kilometres to miles. It arranges these values in the required feature order and applies the saved preprocessor before calling the model's probability-prediction function.

The API returns the model score and classifies a flight as "DELAYED" when the score is at least 0.5; otherwise, it returns "ON_TIME". The response includes historical route and departure-hour delay rates for context. Live flight information from Aviationstack is separate and is not included in the model inputs.

The score is interpreted as a model-generated departure-delay risk estimate rather than a guarantee. The final notebook uses an 80/20 stratified random train/test split with random_state = 42, giving 5,503,588 training rows and 1,375,897 test rows. Hyperparameter selection uses RandomizedSearchCV with 10 candidates, three-fold cross-validation and ROC-AUC scoring on a stratified 300,000-row sample from the training set. The selected model is then fitted on the full training set. A positive-class weight of approximately 3.6095 addresses class imbalance. The recorded selected settings are 500 estimators, maximum depth 8, learning rate 0.03, subsample 0.7 and column-sampling fraction 0.7.

### 5.5 Recorded Evaluation Results

The model artefacts include the following recorded evaluation results:

| Metric | Accuracy | Precision | Recall | F1 Score | ROC-AUC |
|---|---|---|---|---|---|
| Recorded Result | 64.64% | 33.95% | 66.61% | 44.97% | 0.712 |

On 11 October 2026 at 04:55 UTC (12:55 Singapore time), a Colab run reevaluated the saved model and preprocessor without retraining or refitting. It reproduced the original metrics on 1,375,897 held-out flights: 1,077,409 on-time and 298,488 delayed. The confusion matrix is [[690502, 386907], [99659, 198829]], with delayed flights as the positive class. Results, execution output and input hashes are recorded in

evidence/model_evaluation/. This confirms the evaluated exports; matching them to the live deployment remains unverified.

The executed always-on-time baseline achieved 78.31% accuracy and zero delayed-class precision, recall and F1. The model achieved 64.64% accuracy and 66.61% delayed-class recall: it identifies approximately two-thirds of delayed departures, but only one-third of its delay predictions are correct. It trades lower overall accuracy for delayed-flight detection, with substantial false alarms. Probability calibration has not been demonstrated.

### 5.6 Live flight-tracking feature (non-ML)

In addition to the historical risk model, FlyWise integrates a third-party live flight-status API to show current status (on-time, delayed, cancelled, live gate/ETA) for flights that are already scheduled and within the provider's tracking window. This feature is entirely separate from the ML pipeline: it does not feed into or improve the delay-risk model, and it is only available for near-term flights where live data exists. It is included to give users a single place to both plan ahead (via the risk score) and check status close to departure, without implying the two data sources are combined or that live data improves prediction accuracy.

### 5.7 Limitations

The model estimates flight-delay risk from historical patterns rather than real-time operational causes. Its outputs should not be interpreted as guaranteed outcomes, and probability calibration has not yet been demonstrated.

Prediction requires coordinates for both airports. The documented airport-coordinate file covers approximately 79 airports, while Aviationstack can return additional airports. Requests involving unsupported airports return HTTP 422. A flight appearing in the Live Flights page therefore does not guarantee that delay-risk prediction is available.

The live flight-tracking feature depends on the provider's coverage and availability. It is separate from the prediction model and does not improve the model's risk estimate.

The 6,879,485 eligible modelling rows were evaluated using a stratified random holdout, which does not establish future-date performance or applicability outside the dataset. Tuning used preprocessing fitted on the full training split before cross-validation, so its cross-validation score is not a fully isolated preprocessing evaluation. Held-out rows were transformed without refitting. Saved-model reevaluation is complete; full training reproduction, deployed-model consistency, temporal evaluation and calibration remain unverified.

## 6. Testing, Scalability/Resilience and Monitoring Results

### 6.1 Test 1 - Functional test

- Objective: Verify responses from the deployed application's root and delay-causes endpoints, and check that the saved-flight count displayed on the frontend matches the database record count. Direct instance requests do not establish a complete workflow through the ALB.
- Setup: Two EC2 instances running the FlyWise Docker container behind the ALB, connected to the RDS MariaDB instance.
- Steps: Sent GET / and GET /analytics/delay-causes requests directly to a deployed instance; separately, saved flights through the website and queried the upcoming_saved_flights table directly via the running container to compare against what the Prediction page displayed.
- Expected result: HTTP 200 responses; saved-flight count displayed on the frontend matches the database record count.
- Actual result: GET/returned HTTP 200 (~0.032s); GET /analytics/delay-causes returned HTTP 200 (~0.303s). The database contained 8 records in upcoming_saved_flights, matching the Prediction page's displayed "Saved (8)" count after a known frontend list-rendering bug (duplicate React keys causing stale rows to persist across tab switches) was identified and fixed.
- Date: 3 Oct 2026

- Artefact: evidence/test-functional.md (endpoint response logs, saved-flight count verification)

### 6.2 Test 2 - Security test

- Objective: Verify that the database is not directly reachable from outside the application tier, and identify any gaps against least-privilege access.
- Setup: RDS MariaDB instance with public access disabled, SG restricting port 3306 to the EC2 security group.
- Steps: Confirmed RDS "Public access: No" setting; verified SSL enforcement (require_secure_transport ON); ran SELECT CURRENT_USER(), USER() and SHOW GRANTS FOR CURRENT_USER() to audit the application's actual database privileges.
- Expected result: Database is unreachable from outside the VPC/app tier; application connects with a scoped, least-privilege account.
- Actual result: Configuration inspection showed RDS public access disabled, the database security group restricted to the application tier and SSL enforcement enabled. A rejected external-connection test was not recorded, so these observations establish inspected settings rather than demonstrated network rejection. The privilege audit showed that the application connects as admin@% with administrative privileges, including CREATE USER, GRANT OPTION, DROP and ALTER. Least privilege was therefore not achieved; remediation and retesting remain necessary.
- Date: 3 Oct 2026
- Artefact: evidence/test-security.md (grants output, redacted RDS configuration)

### 6.3 Test 3 - Data/AI validation test

- Objective: Reevaluate the saved XGBoost model on the original held-out split and compare an executed always-on-time baseline; inspect the historical dataset's recorded quality checks.
- Setup: flight_data_final.csv, trusted saved model/preprocessor/feature list, and analytics/evaluation_config.json. Colab used Python 3.13.16, pandas 2.2.3, NumPy 2.1.3, scikit-learn 1.6.1 and XGBoost 3.4.1. Input and artefact hashes are recorded in evidence/model_evaluation/run_summary.json.
- Steps: Remove exact duplicates; exclude cancelled/diverted flights; parse dates and HH:MM/HHMM schedules; derive the original 11 features; exclude missing targets; recreate the stratified 80/20 split with seed 42; transform with saved preprocessing; score the saved model at threshold 0.5; calculate metrics and the always-on-time baseline. No retraining or preprocessing refit. Repository-root command: python analytics/evaluate_model.py --data "<path to flight_data_final.csv>" --artifacts "src/ML Model" --feature-spec analytics/evaluation_config.json --out evidence/model_evaluation. Install analytics/requirements-evaluation.txt first.
- Expected result: Reproduce the original eligible/train/test counts and saved-model metrics, and quantify delayed-class detection and false alarms against the baseline.
- Actual result: The run processed 7,001,619 input rows, retaining 6,879,485 eligible rows: 5,503,588 training and 1,375,897 test flights. Model metrics and confusion matrix matched the original values in Section 5.5; the executed baseline achieved 78.31% accuracy and zero delayed-class recall/F1. Saved EDA outputs separately record 27 columns, zero exact duplicates, 98,591 missing departure-delay labels, 122,135 missing arrival-delay labels and two missing scheduled-duration values. The EDA checks were inspected, not freshly rerun.
- Date: Saved-model reevaluation: 11 October 2026, 04:55 UTC. Original model evaluation: 30 September; recorded EDA checks: 10 October 2026.
- Artefacts: evidence/test-data-ai.md; evidence/model_evaluation/metrics.json, baseline_metrics.json, confusion_matrix.csv, run_summary.json and execution.log; analytics/evaluate_model.py, evaluation_config.json, requirements-evaluation.txt and Run_Model_Evaluation_FIXED.ipynb. EDA: analytics/analysis.ipynb.

- Remaining validation: Verify live-deployment artefact consistency; record valid, unsupported-airport and malformed prediction API cases; supply precise dataset acquisition instructions; reconcile analytics/README.md with the notebooks. The completed run reevaluated saved artefacts and did not reproduce model training.

### 6.4 Test 4 - Scalability/resilience test

- Objective: Verify that the configured Auto Scaling Group performs a scale-in action when triggered by its target-tracking policy.
- Setup: Auto Scaling Group (INF2006-FlyWise-ASG) with a target-tracking scaling policy on CPU utilisation (threshold 40%), fronted by the ALB distributing traffic across instances in two Availability Zones.
- Steps: Reviewed the AWS Auto Scaling notification and associated target-tracking alarm information for the recorded scaling event.
- Expected result: The scaling policy reduces desired capacity and records the corresponding instance termination when a scale-in action is triggered.
- Actual result: An AWS notification records a scale-in event reducing desired capacity from 3 to 2 and terminating one instance on 3 October 2026 at 09:35 UTC. This supports an observed scale-in action. The available record does not establish scale-out, response-time behaviour during scaling or uninterrupted service; those outcomes require additional testing and evidence.
- Date: 3 Oct 2026
- Artefact: evidence/test-resilience.md (redacted AWS notification emails, CloudWatch alarm screenshots)

### 6.5 Monitoring results

CloudWatch provides CPU metrics and target-tracking alarms for the 40% CPU scaling policy. The 3 October 2026 notification at 09:35 UTC records desired capacity decreasing from three to two. Monitoring screenshots are indexed in evidence/monitoring.md. They support the documented observation but do not establish response times or uninterrupted service. A complete written record with observation window, metric statistic/unit, alarm condition and interpretation, supported by dated redacted exports, remains outstanding.

## 7. Cost, sustainability and operational considerations

The documented deployment uses EC2 application instances, an ALB, an Auto Scaling Group and RDS MariaDB, with a private S3 bucket holding an approximately 898.8 MB historical CSV. Cost assessment must include instance runtime and disks, RDS capacity and Multi-AZ configuration, ALB usage, object storage, monitoring, public IPv4 resources allocated, data transfer and external API usage. Account-specific credits or free-tier eligibility have not been verified and should not be assumed. A complete monthly estimate or billing record remains outstanding.

Cost controls include choosing small development instances, limiting unnecessary external API calls and reviewing scaling capacity. The inspected application source contains a 60-second SQL-query cache; this does not establish live-flight API caching or a measured performance improvement. Precomputed delay-cause summaries reduce repeated large-table aggregation, although no energy savings were measured. Operational priorities are consistent application/model versions across instances, documented database provisioning, verified backup settings and evidence capture before decommissioning unused resources.

## 8. Team contribution, ethical considerations and reflection

### 8.1 Team Contribution

| Member | Role | Artefacts/Commits | Test/Evidence Ownership |
|---|---|---|---|
| Chong Xin Huei | Machine Learning and Data Analytics | Developed and evaluated the XGBoost flight-delay prediction model, performed data preprocessing and exported ML files for website integration. | ML validation, performance metrics, confusion matrix, baseline comparison and reproducibility testing. |
| Damien Teh Zhanrong | Report preparation and architecture documentation | Reviewed the report against the project brief and deployment documentation; adapted the system and cloud architecture diagrams; improved diagram labels, connections and presentation. | Architecture-diagram documentation and report-to-evidence consistency review. |
| Daniel Tay Zhu Hao | Frontend UI/UX | Built the React and TypeScript frontend using Vite, Tailwind CSS and Recharts, focusing on UI/UX and backend integration. | Worked on the dashboard, prediction form and panel, analytics page, filter panel, data table, header, and shared styling. Added account management. |
| Wong Jinghong | Backend and Database, Web and API Integration | React frontend and FastAPI backend, JWT Authentication and CRUD operations. Integrated Aviationstack's API Key and ML model to retrieve live flights and make delay predictions. | Website functional & non-functional tests, JWT user authentication, React & FastAPI endpoint tests. |
| Wong Zheng Sheng, Jasper | AWS Cloud Deployment | Artefacts: AWS infrastructure setup & troubleshooting, EC2 ASG, ALB, Instances, Launch Templates, RDS Multi-AZ, VPC & Subnets (2 Private + 2 Public), email notifications. | AWS Creation ID, Website functional test, email notification screenshots, IP address and cloud configurations. |

### 8.2 Ethical considerations and responsible use

FlyWise is intended to help travellers interpret historical flight-delay patterns when preparing for a trip. Predictions are estimates rather than guaranteed outcomes and should support a user's judgement about travel arrangements. Comparisons between airlines or routes require care because dataset coverage, sample sizes and operating conditions may differ. The separate live flight-tracking feature provides current information where available and does not improve the historical prediction model.

The historical dataset describes public flight operations rather than passenger-identifying records. User accounts and saved flights introduce privacy responsibilities: collect only necessary information, restrict each

user's access to their own records and redact identifying details from evidence. The submission must exclude credentials, tokens and sensitive account data. Probability calibration and confidence messaging for routes with limited data remain unverified.

### 8.3 Reflection

**Chong Xin Huei:** Through this project, I learnt how machine learning can be integrated into a cloud-based application. Developing the XGBoost model strengthened my understanding of data preprocessing, model evaluation and deployment. I also realised the importance of reproducibility and testing to ensure reliable predictions beyond model accuracy alone.

**Damien Teh Zhanrong:** Through this project, I learnt how to distinguish application architecture from cloud deployment architecture. The system diagram explains how the frontend, backend, prediction model, database and external API interact, while the cloud diagram explains how AWS services support deployment, scaling and availability. Updating these diagrams required checking the implementation details rather than including services simply because they appeared in an earlier design.

A key difficulty was reconciling the report with deployment notes and repository artefacts. I learnt that a configured service or recorded metric does not, by itself, prove security, resilience or predictive quality. Claims need supporting configuration, repeatable tests and dated results. In future projects, I would maintain an evidence checklist alongside development so that documentation and validation remain consistent with the implemented system.

**Daniel Tay Zhu Hao:** Through this project, I learnt how to structure a React and TypeScript application while working on the frontend. On the cloud infrastructure side, deploying FlyWise on AWS showed me how the parts of a cloud system depend on each other. The database runs on Amazon RDS, so the application never hosts its own data. The FastAPI backend is built as a Docker image and runs on EC2 inside our VPC.

**Wong Jinghong:** Through this project, I gained a better understanding of how cloud computing concepts and technologies work together to build a complete, data-driven web application. I learnt how to process large flight datasets, design relational databases, use SQL for flight analytics, develop a React frontend and FastAPI backend, and implement JWT authentication and CRUD functionalities. I also gained experience integrating Aviationstack's API and a machine learning model to provide live flight information and delay predictions. Most importantly, I learnt the importance of security, scalability, data integrity, and cloud deployment. These skills can be applied to real-world applications in banking, e-commerce, transportation, and other data-driven industries.

**Wong Zheng Sheng, Jasper:** Through this project, I gained a better understanding of how different AWS cloud services can come together to create a complete and functional website application. Personally for me, instead of focusing on creating an individual programming component, I learnt how to integrate the frontend, backend, database and cloud infrastructure together to work as one system. I also learnt how services like EC2, ASG, ALB and RDS can be combined together to improve availability and scalability of the website. Through testing and troubleshooting the deployment, I realised that an application that works locally can still encounter issues when deployed to a cloud environment, particularly with networking, database connectivity, security groups and external API access. This helped me develop a more systematic approach to identifying and resolving problems instead of assuming that an issue was caused by the application code itself. Overall, this project improved my technical and problem-solving skills and gave me greater confidence in applying cloud computing concepts to future real-world applications.

## 9. References


1) SkyRefund. (2026, January 30). SkyRefund analysis reveals US travellers lost 1.5 million hours to flight delays in 2025 — and what caused them. PR Newswire. https://www.prnewswire.com/news-releases/skyrefund-analysis-reveals-us-travelers-lost-1-5-million-hours-to-flight-delays-in-2025--and-what-caused-them-302675101.html
2) Morris, C. (2026, May 21). Nearly 1 in 4 flights did not arrive on time last year — and 2026 could be even worse. Here are the worst offenders. Yahoo Finance. https://finance.yahoo.com/economy/articles/nearly-1-4-flights-did-114000062.html

## Appendix A: Dataset field-selection decisions

| Column | Keep/Delete | Reason |
|---|---|---|
| Year | Delete | Derivable from FlightDate; redundant for storage. |
| Quarter | Delete | Can easily be derived from FlightDate or Month; redundant. |
| Month | Delete | Derivable from FlightDate; redundant for storage. |
| DayofMonth | Delete | Derivable from FlightDate; redundant for storage. |
| DayOfWeek | Delete | Derivable from FlightDate; redundant for storage. |
| FlightDate | Keep | Essential. Tells you when the flight occurred and allows filtering by date. |
| Reporting_Airline | Keep | Identifies the airline; important for website searches and airline-delay statistics. |
| Flight_Number_Reporting_Airline | Keep | Identifies the flight number. Useful if users search for a specific flight. |
| Origin | Keep | Origin airport code identifies the departure airport; city names alone do not uniquely identify airports. |
| OriginCityName | Keep | Gives readable city names instead of only airport codes. Useful for frontend display. |
| OriginState | Delete | Omitted from the documented field-selection plan; location display uses the retained city field. |
| Dest | Keep | Destination airport code identifies the arrival airport; city names alone do not uniquely identify airports. |
| DestCityName | Keep | Readable destination city for frontend display. |
| DestState | Delete | Omitted from the documented field-selection plan; destination display uses the retained city field. |
| CRSDepTime | Keep | Scheduled departure time. Very important because this is known before the flight. |
| DepTime | Keep | Actual departure time. Useful for historical comparison with scheduled departure. |

| | | |
|---|---|---|
| DepDelay | Keep | Actual delay in minutes; can include negative values for early departures. Useful for analytics. |
| DepDelayMinutes | Delete | Repeat of DepDelay but negative values are 0 here |
| DepDel15 | Keep | Indicates whether departure was delayed by at least 15 minutes. Used as the final model's target, renamed delayed for training. |
| TaxiOut | Delete | Time spent taxiing from gate to takeoff. Not needed for prediction, but could support airport operational analytics. |
| WheelsOff | Delete | Actual takeoff time. Useful for historical operational analysis. |
| WheelsOn | Delete | Actual landing time. Useful for historical flight analysis. |
| TaxiIn | Delete | Time between landing and reaching the gate. Could be useful for airport/route analytics. |
| CRSArrTime | Keep | Scheduled arrival time. Important for website flight information. |
| ArrTime | Keep | Actual arrival time. Useful for showing historical scheduled-vs-actual performance. |
| ArrDelay | Keep | Actual arrival delay, including possible early arrival values. Useful for analytics. |
| ArrDelayMinutes | Delete | Repeat of ArrDelay but negative values are 0 here |
| ArrDel15 | Keep | Indicates whether arrival was delayed by at least 15 minutes. Retained for historical analytics; it is not the final model's training target. |
| Cancelled | Keep | Important flight outcome. Lets you calculate cancellation rates and distinguish cancelled flights from delayed flights. |
| CancellationCode | Keep | Gives the reason/category for cancellation. Mostly missing because most flights were not cancelled; that is expected. |
| Diverted | Keep | Important flight outcome showing whether the aircraft was diverted. |

| | | |
|---|---|---|
| CRSElapsedTime | Keep | Scheduled gate-to-gate journey duration. Useful for route information and later ML. |
| ActualElapsedTime | Keep | Actual gate-to-gate duration. Useful for historical scheduled-vs-actual analysis. |
| AirTime | Keep | Actual time spent airborne. Could support route efficiency/duration analytics. |
| Distance | Keep | Route distance in miles; retained for historical analytics and used by the final model. |
| CarrierDelay | Keep | Indicates how much delay was attributed to the airline/carrier. Useful for delay-cause analytics. |
| WeatherDelay | Keep | Useful for analysing delays attributed to weather. |
| NASDelay | Keep | Delay attributed to the National Airspace System, such as traffic or airport operations. Useful for cause analysis. |
| SecurityDelay | Keep | Delay attributed to security issues. Useful even though relatively uncommon. |
| LateAircraftDelay | Keep | Delay caused by the aircraft arriving late from a previous flight. Useful for explaining historical delay causes. |